

Implementation Report

Date: July 27, 2026

Lead Engineer: Ashadapu Ranjith Kumar

Project: CARIVIX-AI

1. Objective & Scope

The primary objective for today's implementation was to migrate the WebGIS visualizer from a static file-based architecture to a dynamic RESTful API service using FastAPI. This required updating client-side data sources, fixing UI rendering blocks and HTTP 404 routing errors, and optimizing spatial payload delivery for large GeoJSON datasets.

2. Code Changes & Architecture Implementation

A. FastAPI Server Implementation (server.py)

Developed server.py to handle both application interface rendering and spatial GeoJSON endpoint delivery.

```
from fastapi import FastAPI, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.gzip import GZipMiddleware
from fastapi.responses import FileResponse, JSONResponse
import os
import json

app = FastAPI(
    title="India Administrative Boundary & Point GIS API",
    description="High-performance WebGIS spatial data delivery API serving GADM v4.1 administrative boundary vectors and spatial density point layers.",
    version="1.0.0"
)
```

```
# Enable CORS for external client access
```

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["*"],  
    allow_methods=["GET"],  
    allow_headers=["*"],  
)
```

```
# Enable Gzip compression (70%-80% compression on JSON responses > 1KB)
```

```
app.add_middleware(GZipMiddleware, minimum_size=1000)
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
BOUNDARIES_MAP = {  
    0: "assets/optimized_geojson/gadm41_IND_0_opt.geojson",  
    1: "assets/optimized_geojson/gadm41_IND_1_opt.geojson",  
    2: "assets/optimized_geojson/gadm41_IND_2_opt.geojson",  
    3: "assets/optimized_geojson/gadm41_IND_3_opt.geojson"  
}
```

```
@app.get("/", summary="Render WebGIS Application UI", tags=["Frontend Application"])
```

```
async def render_map():
```

```
    index_path = os.path.join(BASE_DIR, "index.html")
```

```
    if not os.path.exists(index_path):
```

```
        raise HTTPException(status_code=404, detail="index.html not found.")
```

```
    return FileResponse(index_path, media_type="text/html")
```

```
@app.get("/api/v1/boundaries/{level}", summary="Get Boundary GeoJSON Payload",
tags=["Administrative Boundaries"])
```

```
async def get_boundary(
```

```
    level: int,
```

```
    state: str = Query(None, description="Optional State filter to reduce payload size (e.g., 'Telangana')")
```

```
):
```

```
    if level not in BOUNDARIES_MAP:
```

```
        raise HTTPException(status_code=400, detail="Invalid boundary level. Choose 0, 1, 2, or 3.")
```

```
    file_path = os.path.join(BASE_DIR, BOUNDARIES_MAP[level])
```

```
    if not os.path.exists(file_path):
```

```
        raise HTTPException(status_code=404, detail=f"Boundary file level {level} not found on server.")
```

```
    # In-memory feature filtering for query parameters
```

```
    if state and level > 0:
```

```
        with open(file_path, "r", encoding="utf-8") as f:
```

```
            data = json.load(f)
```

```
            filtered_features = [
```

```
                feat for feat in data.get("features", [])
```

```
                if feat.get("properties", {}).get("NAME_1", "").lower() == state.lower()
```

```
            ]
```

```
            return JSONResponse(content={"type": "FeatureCollection", "features": filtered_features})
```

```
    return FileResponse(file_path, media_type="application/geo+json")
```

```
@app.get("/api/v1/points/sample", summary="Get Spatial Sample Points & Weights", tags=["Point & Heatmap Data"])
```

```
async def get_sample_points():  
  
    file_path = os.path.join(BASE_DIR, "assets/points/sample_points.geojson")  
  
    if not os.path.exists(file_path):  
  
        raise HTTPException(status_code=404, detail="Sample points dataset not found.")  
  
  
    return FileResponse(file_path, media_type="application/geo+json")
```

B. Client Application Integration (index.html)

Replaced hardcoded relative file paths (./assets/optimized_geojson/...) in MapLibre GL sources with live API routes.

```
map.on('load', () => {  
  
    // Level 3: Sub-Districts Endpoint  
  
    map.addSource('subdistricts', {  
  
        type: 'geojson',  
  
        data: 'http://localhost:8000/api/v1/boundaries/3'  
  
    });  
  
  
    // Level 2: Districts Endpoint  
  
    map.addSource('telangana-districts', {  
  
        type: 'geojson',  
  
        data: 'http://localhost:8000/api/v1/boundaries/2'  
  
    });  
  
  
    // Level 1: States Endpoint  
  
    map.addSource('state-boundaries', {  
  
        type: 'geojson',  
  
        data: 'http://localhost:8000/api/v1/boundaries/1'  
  
    });
```

```
// Heatmap & Point Data Endpoint

map.addSource('sample-points', {

  type: 'geojson',

  data: 'http://localhost:8000/api/v1/points/sample'

});

});
```

3. Issues Identified & Technical Resolutions

Component	Error / Symptom	Root Cause	Technical Resolution
Web Root (/)	{"detail": "Not Found"}	Missing root route handler in FastAPI script.	Added @app.get("/") returning FileResponse("index.html").
Swagger UI (/docs)	Infinite loading / UI freezing on Level 2 & 3 boundary execution.	Browser DOM memory limits when parsing and syntax-highlighting raw 16.7MB+ GeoJSON responses.	Added GZipMiddleware for transit compression and implemented server-side state-level filtering (?state=Telangana).
Payload Delivery	High latency on spatial boundary network transfers.	Serving uncompressed, raw text GeoJSON.	Integrated FastAPI Gzip middleware to reduce network transfer payload sizes by ~70%–80%.

4. Verification & Testing

1. **Application UI Mounting:** Verified that opening <http://localhost:8000/> loads the MapLibre map canvas along with controls and layer overlays.
2. **OpenAPI Interface:** Verified that <http://localhost:8000/docs> generates working interactive documentation for all routes.
3. **Filtered Requests:** Executed filtered spatial queries (e.g., </api/v1/boundaries/2?state=Telangana>) and confirmed instantaneous payload response without browser hang.